CSE 753: Advanced Reinforcement Learning

Study Notes — Policy Gradient Methods (Lecture 06) and Actor-Critic Methods (Lecture 07)

1 Lecture 06: Policy Gradient Methods

1.1 From value tables to a direct policy (Slides 2–3)

In earlier lectures, reinforcement learning worked like this: start with some policy, then keep repeating two steps forever — first figure out how good the current policy is (by estimating $Q(s, a)$ or $V(s)$), then use those numbers to make the policy better (for example, always pick whichever action has the highest Q-value). This means you have to keep track of a value for every single state-action pair, which is like keeping a giant scoreboard for every possible situation.

Slide 3 asks a different question: what if we skip the scoreboard completely, and just directly train the policy itself? We give the policy a bunch of numbers to control it, called $\vec{\theta}$ (think of $\vec{\theta}$ as a row of dials, like the knobs on a mixing board), and write

$$\pi(a \mid s, \vec{\theta}) = \Pr\{A_t = a \mid S_t = s, \vec{\theta}_t = \vec{\theta}\}.$$

Symbol	What it means
$\vec{\theta}$	that list of dial-settings.
$\pi(a \mid s, \vec{\theta})$	the chance that, with the dials set to $\vec{\theta}$, the policy chooses action a while it is in state s.

So this equation is just a definition: it's naming "the probability of taking action a in state s, given the current dial settings" as $\pi(a \mid s, \vec{\theta})$.

Aspects of "direct policy parameterization / policy gradient methods."

- **Advantage:** it works well even when there are a huge number of actions, or the actions are continuous (like steering angles), because you never need to search through every possible action to find the best Q-value. It also directly trains the exact thing you will use in the end — the policy — and it can naturally act in a randomised (not always-the-same) way when that is useful.
- **Disadvantage:** the dial-nudging direction has to be guessed from a handful of sample runs, so the guess is noisy and can wobble around a lot; it needs many tries; it can get stuck doing something that is only okay, not great.
- **Problem it solves:** Old methods had to check every single action and ask "how good is this one?" before picking the best one. That's fine if you only have a few choices, like picking a flavor from 5 ice creams. But if you have thousands of choices, or the choice is something like "turn the steering wheel by exactly this many degrees" (infinite options), checking every single one becomes impossible. Policy gradient skips this entirely — it trains the policy to just directly spit out a good action, so there's nothing to search through.

- **Problem it cannot solve** Just switching to policy gradient doesn't fix two other headaches. First, since it learns from a small number of sample runs, its guess about which way to adjust the dials is shaky and jumps around a lot (this is the "wobbly guess" problem). Second, if a whole run turns out badly, policy gradient can't tell which specific action in that run actually caused the bad result — it just blames everything equally. Both of these problems still need to be fixed with extra tricks covered later.
- **Problems it gives rise to:** the most basic version of this idea, called REINFORCE, turns out to be noisy, bad at giving credit to the right action, and only works with fresh data from the very latest dial settings. Almost everything else in these two lectures is about fixing one of those three problems.
- **When to use it:** when there are many possible actions, or the actions are continuous, or you actually want some randomness in the behaviour.
- **When not to use it:** when there are only a few actions to choose from and a simple value-based method already works fine — adding randomness would not help there.
- **What would change if done differently:** if we had kept picking actions by searching through Q-values instead, we would run right back into the exact scaling problems that made us want a different approach in the first place.

1.2 Parameterizing the policy: softmax (Slide 4)

For $\pi(a \mid s, \theta)$ to be a real, valid policy, two things must be true: it can never be negative, $\pi(a \mid s, \theta) \geq 0$, and for any state s, the probabilities of all the actions must add up to exactly 1, i.e. $\sum_{a \in A} \pi(a \mid s, \theta) = 1$. This is just saying "it has to behave like a real set of chances," the same way a fair coin has a 50% chance of heads and 50% chance of tails, adding up to 100%.

One simple way to build such a policy is the **softmax** function:

$$\pi(a \mid s, \theta) = \frac{e^{h(s,a,\theta)}}{\sum_{b \in A} e^{h(s,b,\theta)}}.$$

Symbol	What it means
$h(s, a, \theta)$	just a raw score (sometimes called a "preference") that the current dial settings θ give to action a in state s — think of it as an opinion score that can be any number, even negative.
$e^{(\cdot)}$	the exponential function; all it does here is turn any number, positive or negative, into a positive number.
$\sum_{b \in A} e^{h(s,b,\theta)}$	the bottom of the fraction. It adds up these positive, exponentiated scores for every possible action b, so we can divide by that total.

So the whole recipe is: give every action a raw opinion score, turn every score into a positive number, then divide by the sum of all of them so everything adds up to exactly 1. That turns arbitrary scores into real probabilities. If one action's score becomes much bigger than the rest, it grabs almost all of the probability, so the policy starts acting almost like it always picks that one action (we call this "greedy" behaviour) — even though, technically, there is still a tiny chance of picking something else.

1.3 Why stochastic policies matter (Slide 5)

The slide shows a small grid of boxes: Start (S), then two middle boxes, then Goal (G), and every step costs $R = -1$ (like paying a toll for each move). Here's the twist: because the policy uses a simplified way of looking at the world (called function approximation), the two middle boxes end up looking exactly the same to the policy, even though they are really different situations. One of them needs "go right" to make progress, and the other needs a different move to avoid getting stuck going back and forth. Since the policy cannot tell the two boxes apart, no single fixed action can be correct in both of them. The slide shows that the best the policy can do is act randomly: go left 41% of the time and right 59% of the time, written as $41\% \leftarrow \pi_\theta \rightarrow 59\%$, which gives an expected long-run score of $V_{\pi_\theta}(s) = -11.6$.

In plain words: if two genuinely different situations look identical to your policy, then any policy that always does the exact same thing in that situation will be wrong at least some of the time. Mixing two different actions with some probability lets the policy do reasonably well across both hidden situations on average. That is why, here, a bit of randomness beats always doing the same fixed thing.

Aspects of "stochastic vs. deterministic policies."

- **What it is:** a stochastic policy picks actions with some randomness (a chance for each action); a deterministic policy always outputs one fixed action.
- **Advantage:** when the policy can't fully tell two situations apart, the best stochastic policy can genuinely beat every possible deterministic policy, as shown by the -11.6 example above. It also helps the agent explore instead of getting stuck doing the same thing forever.
- **Disadvantage:** it adds unpredictability, which can be a problem in situations where you truly need one reliable, repeatable action; it also makes the behaviour harder to test and double-check.
- **Problem it solves:** it rescues performance when the way we represent the state hides some real difference between situations.
- **Problem it cannot solve:** it does not fix the underlying issue of the state representation losing information — the policy is just making the best of a bad hand it was dealt.
- **When to use it:** whenever the state you show the policy might be hiding useful details, or when you need the agent to explore.
- **When not to use it:** when the state already tells the policy everything it needs to know, and one best action truly exists and is knowable — adding randomness there would only make things worse.

1.4 The RL objective (Slide 6)

$$\vec{\theta}^* = \arg\max_\theta \mathbb{E}_{\tau\sim p_\theta(\tau)}[r(\tau)] = \arg\max_\theta \underbrace{\mathbb{E}_{\tau\sim p_\theta(\tau)}\Big[\sum_t r(s_t, a_t)\Big]}_{J(\theta)}, \qquad J(\theta) \approx \frac{1}{N}\sum_i \sum_t r(s_{i,t}, a_{i,t}).$$

Symbol	What it means
τ	a trajectory: one full run of the task, from the very first state to the very last, including every action taken along the way.
$p_\theta(\tau)$	the probability that this particular run happens, if you are using the policy with dial settings θ.
$r(\tau)$	the total reward earned over that whole run, which is just $\sum_t r(s_t, a_t)$ added up over every timestep t.
$\mathbb{E}_{\tau\sim p_\theta}[\cdot]$	"the average of whatever is inside the brackets, if you ran the policy over and over and over."
N	how many runs we actually collected (by simulating or acting in the real world).
i	which run we are talking about.
t	which timestep inside that run.
θ^*	the best dial setting we are searching for.

In plain words: we want to find the dial settings that make the average total reward, over many runs of the task, as large as possible. We can't actually compute that true average exactly, since that would mean knowing every possible run and its exact chance of happening. So instead, we do the next best thing: run the policy N times, add up the reward from each run, and take the average. That average, called $J(\theta)$, stands in for the real objective we actually care about.

1.5 Deriving the policy gradient (Slides 7–8)

Slide 7 starts from $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[r(\tau)] = \int p_\theta(\tau)\, r(\tau)\, d\tau$ and works out its gradient (the direction that increases it fastest):

$$\nabla_\theta J(\theta) = \int \nabla_\theta p_\theta(\tau)\, r(\tau)\, d\tau = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, r(\tau)\, d\tau = \mathbb{E}_{\tau\sim p_\theta}\big[\nabla_\theta \log p_\theta(\tau)\, r(\tau)\big],$$

using the fact that $p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau)$ (this is just basic calculus rearranged, since $\nabla_\theta \log p_\theta(\tau)$ is defined as $\nabla_\theta p_\theta(\tau)$ divided by $p_\theta(\tau)$).

Symbol	What it means
$\int(\cdot)\, d\tau$	"add this up over every single possible run" — since there are infinitely many possible runs, we use an integral instead of a simple sum.
∇_θ	"the gradient with respect to the dial settings θ": a list of directions, one per dial, telling us which way to turn each dial to make something bigger.
$\log p_\theta(\tau)$	the logarithm of how likely that run was.

In plain words: we want the gradient of J so we know exactly which way to turn the dials to get more reward. The trouble is, $\nabla_\theta p_\theta(\tau)$ is not something we can estimate just by running the policy and watching what happens — it isn't itself an average of anything we can sample. The trick on this slide rewrites the gradient of the probability as "the probability itself, times the gradient of its logarithm." That trick matters because now the whole expression looks exactly like "probability times something," which is exactly what an average over sampled runs computes. So now we *can* estimate it, just by running the policy many times and watching what happens.

Slide 8 breaks $\log p_\theta(\tau)$ into pieces, using $p_\theta(\tau) = p(s_1)\prod_{t=1}^{T}\pi_\theta(a_t \mid s_t)\,p(s_{t+1} \mid s_t, a_t)$:

$$\nabla_\theta \log p_\theta(\tau) = \nabla_\theta\Big[\log p(s_1) + \sum_{t=1}^{T}\log \pi_\theta(a_t \mid s_t) + \sum_{t=1}^{T}\log p(s_{t+1} \mid s_t, a_t)\Big].$$

Symbol	**What it means**
$p(s_1)$	the chance of starting out in state s_1 — this is decided by the environment, not by our dial settings θ.
$\pi_\theta(a_t \mid s_t)$	the policy's own chance of taking the action it took, which *does* depend on θ.
$p(s_{t+1} \mid s_t, a_t)$	the chance that the world moves to the next state, given the current state and action — this is decided by the physics or rules of the world, not by θ either.

In plain words: the chance of one whole run happening breaks into three pieces — where you started, what the policy chose to do at every step, and how the world reacted at every step. When we take the gradient with respect to θ, the "where you started" piece and the "how the world reacted" piece both disappear completely, because they don't depend on θ at all — only the "what the policy chose" piece survives. This is exactly why policy gradient methods don't need a model of the world: we never had to know or work with the world's own rules, $p(s_{t+1} \mid s_t, a_t)$, at all.

Putting the pieces together, the final gradient (still Slide 8) becomes:

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\Big(\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})\Big)\Big(\sum_{t=1}^{T} r(s_{i,t}, a_{i,t})\Big)$$

Symbol	**What it means**
N	the number of runs (trajectories/episodes) you actually collected by running the policy. This is a number you choose, like "collect 32 episodes before each update."
$s_{i,t}$, $a_{i,t}$	the specific state and action that occurred at timestep t, inside run i. The double subscript pins down "which run" and "which moment in that run" — e.g., $s_{3,7}$ means "the state at step 7 of run 3."
$\sum_{t=1}^{T}\nabla_\theta \log \pi_\theta(a_{i,t} \mid s_{i,t})$	for a single run i, add up the gradient contribution from every timestep in that run. Each term is a list of numbers, one per dial in θ, pointing toward making the action actually taken at that step more likely next time. Summing over t combines these "make this more likely" directions from every step into one overall direction for that run.
$\sum_{t=1}^{T} r(s_{i,t}, a_{i,t})$	for that same run i, add up the actual reward received at every timestep. This gives a single number: the total reward that run earned, start to finish.

In plain words: for every run we sampled, we work out two things — a direction that would make everything the policy did in that run more likely, and a single number saying how good that run's total reward was. We multiply the direction by the reward number, and then average this

over all our sampled runs. Runs with a big total reward push the dials hard in the direction of "do this again." Runs with a low or negative total reward push the dials away from what was done in them.

1.6 REINFORCE: the full vanilla algorithm (Slide 9)

1. Run the current policy $\pi_\theta(a_t \mid s_t)$ to collect a set of runs, $\{\tau^i\}$.
2. Estimate the gradient: $\nabla_\theta J(\theta) \approx \sum_i \left(\sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i)\right)\left(\sum_t r(s_t^i, a_t^i)\right)$.
3. Update the dials a little bit: $\vec{\theta} = \vec{\theta} + \alpha \nabla_\theta J(\theta)$, where α is the learning rate — a small number that controls how big a step we take.
4. Repeat until the policy stops improving.

This is exactly the formula we just built, turned into a loop you can actually run: collect some data, estimate which way to turn the dials, turn them a little bit, and repeat.

1.7 Understanding the gradient intuitively (Slide 10)

In plain words, the whole gradient formula boils down to this: make the actions you took in high-reward runs more likely to happen again, and make the actions you took in low-reward (or negative-reward) runs less likely. As the slide puts it: *do more of the good stuff, less of the bad stuff.*

Aspects of "REINFORCE / vanilla policy gradient."

- **What it is:** the most basic policy gradient method: run the policy, weight each action's "make me more likely" direction by that whole run's total reward, average everything, and take a small step.
- **Advantage:** very simple to understand and implement; it needs no model of the environment; on average, it points in exactly the right direction (statisticians call this "unbiased").
- **Disadvantage:** the direction it estimates wobbles around a lot from batch to batch (high variance); it is sensitive to how big or small the reward numbers happen to be; it gives a whole run's reward as credit to every single action in that run, even ones that had nothing to do with the outcome (poor credit assignment); and it only works with data from the very latest dial settings.
- **Problem it solves:** it lets us improve a randomised policy directly from sampled experience, with no world-model and no scoreboard of values needed.
- **Problem it cannot solve:** it cannot tell you which specific action inside a run actually caused the good or bad outcome. Fixing that is the whole point of the next few slides.
- **Problems it gives rise to:** the failure patterns shown in Slides 11, 13, and 15 below, plus the "must throw away old data" problem in Slide 16.
- **When to use it:** as a starting point, or in cheap, short simulations where a wobbly gradient estimate isn't a big deal.
- **When not to use it:** whenever collecting data is expensive (real robots, real people) or tasks run for a long time — use the fixes described below instead.

1.8 Quiz 1 — the credit-assignment failure (Slide 11)

The setting: a simulated robot is learning to walk. The reward is how fast it moves forward (a negative number if it moves backward instead). Five sample runs include some bad ones (falls backward, marked ×) and some good ones (falls forward or manages to stand still, marked ✓), including run τ_4: "takes one small step forward, then falls backward" — which overall counts as bad (×), since the total reward for that run ends up negative.

Here is the failure: because REINFORCE grades every single action in a run using that *whole* run's total reward, the good first action in τ_4 (the small step forward) gets pushed down right alongside the bad ending, just because they happened to occur in the same run. The gradient has no way to separate "this one action was actually good" from "this whole run turned out badly." This is called a **credit-assignment failure**: the method cannot figure out which action inside the sequence really deserves the credit or the blame.

1.9 Reward-to-go: the fix for credit assignment (Slide 12)

The fix relies on a simple, common-sense fact: what the policy does at time t cannot possibly change a reward that already happened before time t — you can't affect the past. So instead of grading an action using the whole run's reward, we grade it using only what happens *from that point onward*:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) \left(\sum_{t'=t}^{T} r(s_{t'}^i, a_{t'}^i) \right).$$

Symbol	What it means
$\sum_{t'=t}^{T} r(s_{t'}^i, a_{t'}^i)$	called the **reward-to-go**, often written $\hat{Q}_t^i$: it's the sum of all the rewards from timestep t (including t itself) all the way to the very end of the run — not counting anything from before t.

In plain words: instead of grading action t using the whole run (past + future), we grade it using only what came after it (present + future). Since an action truly cannot change rewards that already happened earlier, including those earlier rewards in the grade was only adding noise, never useful information. Removing that noise gives a cleaner, fairer gradient. (As a concrete example, for the table $\tau_1 = (11, 10)$, $\tau_2 = (9, 10)$, $\tau_3 = (0, -1)$, $\tau_4 = (-2, -1)$, the reward-to-go at each timestep is found by adding up rewards starting from that timestep to the end, always including the current step.)

1.10 Quiz 2 — high variance and reward scale (Slide 13)

The setting: the same walking-robot task, but now all four sample runs are successes (✓). A quick flag: the answer text printed on this slide is copied from Slide 11 and doesn't actually fit this new scenario, since every run here is already good — there's no "pushed down a good step" problem to talk about. The slide's own second sentence is the point actually worth remembering: *policy gradient is noisy and sensitive to how big the reward numbers are.* Even when everything succeeds, the raw size of the reward-to-go numbers changes how big a step the gradient takes for each run. If the rewards happen to be big numbers, the steps can be way too large; if they're small numbers, learning can nearly stop. This is exactly the problem that a baseline (up next) is built to fix.

1.11 Baseline: fixing "all returns are the same sign" (Slide 14)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\big[\nabla_\theta \log p_\theta(\tau)\,(r(\tau) - b)\big], \qquad b = \frac{1}{N}\sum_i r(\tau_i).$$

Symbol	What it means
b	called the **baseline**: in this slide, it is simply the average total reward over the N sampled runs.
$(r(\tau) - b)$	the "baselined return" — how much better or worse this particular run did compared to an average run.

Imagine every reward is always somewhere between -1 and -0.5. Even the very best run still gets a raw reward that's negative, so mathematically it would still weakly push in a consistent direction, but this outcome would depend entirely on where we happened to put the zero point of our reward scale, which shouldn't matter at all for learning. Subtracting the average return fixes exactly this: now only runs that did *better than average* get pushed toward "more likely," and runs that did *worse than average* get pushed toward "less likely" — no matter what the raw numeric scale of the rewards happens to be.

Here is the proof that subtracting a constant baseline doesn't secretly change what the gradient is aiming for:

$$\mathbb{E}[\nabla_\theta \log p_\theta(\tau)\, b] = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)\, b\, d\tau = \int \nabla_\theta p_\theta(\tau)\, b\, d\tau = b\,\nabla_\theta \int p_\theta(\tau)\, d\tau = b\,\nabla_\theta(1) = 0.$$

Symbol	What it means
$\int p_\theta(\tau)\, d\tau = 1$	just the plain fact that the chances of every possible run must add up to exactly 1, no matter what θ is — so this quantity is a fixed constant, and its gradient is always exactly 0.

Reading this chain from right to left: the total probability over every run is always 1, no matter how we set θ, so its gradient with respect to θ must be 0. Multiplying that 0 by the constant b still gives 0. Tracing that 0 back through the earlier steps shows it is exactly the extra term that subtracting b adds into the gradient. So subtracting any constant baseline does not change the gradient's true target at all — it stays unbiased — while it can still make the gradient estimate wobble around much less. One important detail: a baseline reduces the wobble, it does not remove it completely.

Aspects of "baselines."

- **What it is:** a number subtracted from the return (or reward-to-go) before it gets multiplied into the gradient direction.
- **Advantage:** it cuts down how much the gradient estimate wobbles, without changing what it's aiming for on average; it also makes the sign of the number (positive or negative) mean something real — above or below average — instead of depending on an arbitrary shift in how rewards are numbered.

- **Disadvantage:** it reduces the wobble but doesn't remove it; a single number like the average return is the same for every state, so it can't tell that some states are simply harder or easier than others. That gap is exactly what leads to using a state-by-state baseline, $V^{\pi}(s)$, in Lecture 07.
- **Problem it solves:** it fixes sensitivity to the reward's numeric scale, and the "everything gets a small push in the same direction" problem when all raw rewards happen to share the same sign.
- **Problem it cannot solve:** when rewards are sparse (see Slide 15), almost every run gets nearly the same reward-to-go, so the baselined number stays nearly the same across samples and learning is still slow. A single scalar baseline also does not fix credit assignment *between different actions inside one run.*
- **Problem it gives rise to (or points toward):** since one fixed number can't adapt to different states, the natural next step is to learn a baseline that changes with the state — a critic, $V^{\pi}(s)$ — which is exactly the actor-critic idea in Lecture 07.
- **When to use it:** basically always — there is no real cost to subtracting a baseline that doesn't introduce bias.
- **When not to use it / limitation:** don't expect a single constant baseline to fix sparse-reward tasks or fine-grained credit assignment by itself.
- **What would change if done differently:** if b were allowed to secretly depend on θ, or on the action itself, in a way not accounted for in the proof above, the "gradient of 1 is 0" trick would break, and the baseline would quietly bias the whole estimate. That is why b must not depend on the action being taken — it can depend on the state, or just be one fixed number.

1.12 Quiz 3 — sparse rewards still cause high variance (Slide 15)

The setting: learning to fold a jacket, where the reward is 0 (not folded), 0.5 (folded but wrinkly), or 1 (folded neatly). Most sample runs score 0 (the jacket wasn't touched, or only the sleeves got folded, or it was flattened but not folded), and only one run scores well. Here's the failure: even after using reward-to-go and a baseline, the gradient's grade is almost the same number (close to zero, after subtracting the average) for nearly every run, because the reward signal is **sparse** — almost nothing in most of the sampled behaviour stands out as clearly better or worse. Policy gradient is still noisy here, not because of scale this time, but because there just isn't enough useful signal to learn from. This is exactly the gap that a learned critic is built to close: instead of waiting for one sparse reward at the very end of an episode, a critic can hand out partial credit along the way, which is the whole motivation for Lecture 07.

1.13 The on-policy problem (Slide 16)

Vanilla policy gradient is called **on-policy**, because the formula's average is only valid for runs τ that come from the *current* dial settings θ. As soon as θ changes even a little, any leftover data collected under the old θ no longer comes from the right source, so it can't be used correctly anymore. In practice, this means you have to throw away all your data and collect a whole new batch after every single small update — which wastes a huge amount of effort. The natural question: can we somehow reuse data collected under an older policy? The answer, coming up next, is off-policy learning using a technique called **importance sampling**.

1.14 Off-policy policy gradient via importance sampling (Slide 17)

First, here's the correction if we try to reuse whole runs from an old policy:

$$\nabla_{\theta'} J(\theta') = \mathbb{E}_{\tau \sim p_\theta(\tau)} \left[\Big(\prod_{t=1}^{T} \frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)} \Big) \Big(\sum_{t=1}^{T} \nabla_{\theta'} \log \pi_{\theta'}(a_t \mid s_t) \Big) \Big(\sum_{t'=t}^{T} r(s_{t'}, a_{t'}) - b \Big) \right].$$

Symbol	What it means
θ	the old policy that actually produced the data.
θ'	the new policy we now want the gradient for.
$\frac{\pi_{\theta'}(a_t \mid s_t)}{\pi_\theta(a_t \mid s_t)}$	called the **importance ratio** at time t: it compares how much more (or less) likely the new policy is to take the exact same action the old policy actually took, in that same state.
$\prod_{t=1}^{T}$	means "multiply all of these ratios together," one for every timestep in the whole run.

In plain words, we have data collected by the old policy π_θ, but we want the gradient for a different, new policy $\pi_{\theta'}$. Importance sampling re-weights the old data: multiply by how much more, or less, likely θ' was to have produced this exact same run, compared to θ. Here is the catch, spelled out right on the slide: multiplying T ratios together makes the correction blow up toward infinity or shrink toward zero *extremely fast* as the run gets longer, or as the new policy drifts away from the old one. For long runs, or a policy that has changed a lot, this correction becomes useless numbers.

The slide then asks: what if, instead of correcting for a whole run at once, we correct one timestep at a time?

$$\nabla_{\theta'} J(\theta') \approx \frac{1}{N} \sum_{i=1}^{N} \sum_{t=1}^{T} \frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_{i,t} \mid s_{i,t}) \left(\sum_{t'=t}^{T} r(s_{i,t'}, a_{i,t'}) - b \right),$$

which the slide calls "much less likely to explode or vanish, but hard to measure," and marks as the final version to actually use. Here, notice that only *one* importance ratio shows up for each timestep — not a whole chain of T ratios multiplied together like before.

In plain words: instead of correcting for how the chance of the *entire* run changed between the old and new policy, we only correct for the mismatch in the action-probability at *this one step*. That is far more stable numerically, since we're never multiplying more than one ratio at a time. But there's a hidden cost, and this is the "hard to measure" part the slide warns about: this shortcut quietly ignores the fact that *which states you even end up visiting* also depends on which policy generated the data. It only fixes the mismatch in action choice at a given state, not the mismatch in which states you land in along the way. In practice, this shortcut is only trustworthy when the new policy θ' hasn't drifted very far from the old policy θ — which is exactly why later methods, especially PPO in Lecture 07, work hard to keep the two policies close together.

1.15 Off-policy policy gradient: full algorithm (Slide 18)

1. Run the current policy $\pi_\theta(a_t \mid s_t)$ to collect a set of runs, $\{\tau^i\}$.
2. Compute $\nabla_\theta J(\theta)$ using $\{\tau^i\}$, with the importance-weighted formula above.
3. Update the dials: $\vec{\theta} = \vec{\theta} + \alpha \nabla_\theta J(\theta)$.

The practical payoff, spelled out on the slide: because the importance ratio corrects for the mismatch between the old and new policy, we can now take **several gradient steps on the very same batch** of data, instead of throwing it away after just one step, the way vanilla REINFORCE had to.

1.16 Bounding how far the policy can move (Slide 19)

Here's a question the slide raises: what if the policy changes by a lot before we go collect fresh data? Then the data we're using no longer reflects the situations that the (now quite different) policy would actually run into, and the gradient estimate becomes inaccurate. This is exactly the hidden cost flagged in the previous slide showing up in practice. The proposed fix: put a limit on how far the policy is allowed to move in a single update.

$$\mathbb{E}_{s\sim\pi_\theta}\left[D_{KL}(\pi_{\theta'}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s))\right] \leq \delta.$$

Symbol	What it means
$D_{KL}(p \,\|\, q)$	a number called the **Kullback-Leibler divergence**. All it does is measure how different two probability distributions p and q are — it's always zero or bigger, and it equals exactly zero only when p and q are the same.
p	the new policy's action choices at state s.
q	the old policy's action choices at that same state.
δ	a small, allowed "budget" for how different the two policies are permitted to be, on average, across the states the old policy tends to visit.

In plain words: on average, across the situations the old policy usually finds itself in, don't let the new policy's choices drift more than δ away from the old policy's choices. By keeping the new policy close to the old one, the situations the new policy will run into stay close to the situations that actually show up in the collected data — which is exactly what keeps the hidden cost from the previous slide small and safe to ignore. This idea is the direct bridge into Lecture 07's methods that either enforce a KL limit, or use a simpler trick (clipping, in PPO) to get almost the same effect.

Aspects of "off-policy policy gradient via importance sampling / KL constraint."

- **What it is:** a way of re-using data from an older policy, by re-weighting each sample according to how much more or less likely the new policy is to have produced it, together with a rule limiting how far the new policy can drift.
- **Advantage:** it lets us reuse data and take several gradient steps per batch, which is a big sample-efficiency win over strictly on-policy REINFORCE.
- **Disadvantage:** correcting for a whole run explodes or vanishes as the run gets longer; the safer per-timestep version quietly drops the correction for which states get visited, so it becomes wrong once the policy drifts too far; and the KL limit itself is expensive and awkward to enforce exactly.
- **Problem it solves:** the "must throw away all data after each step" problem from Slide 16.

- **Problem it cannot solve:** it does not solve the general off-policy problem for data that is very old or very different — it only stays accurate for policies close to the one that generated the data. Data from a large pool of very old experience breaks even this fix, which is why Lecture 07 needs a different tool (learned Q-functions) for that case.
- **Problem it gives rise to:** because exactly enforcing a KL limit is hard, this leads directly to PPO's simpler idea in Lecture 07: instead of limiting the KL divergence exactly, just cap (clip) the importance ratio.
- **When to use it:** when you want a few extra gradient steps out of each batch of fairly fresh, on-policy-like data.
- **When not to use it:** with data from a policy that is very different from the current one, such as an old replay buffer — the correction becomes unreliable there.
- **What would change if done differently:** if we used the whole-run product of ratios instead of the safer per-timestep version, the numbers would blow up or vanish for almost any reasonably long run.

2 Lecture 07: Actor-Critic Methods

2.1 Revisiting value functions and defining Advantage (Slide 2)

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}[Q^\pi(s,a)]$$
$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

Symbol	What it means
$V^\pi(s)$	the value function: the total future reward you expect, on average, starting in state s and then just following policy π from there on.
$Q^\pi(s,a)$	the action-value function: the total future reward you expect if you start in state s, take action a right now, and then follow π afterward.
$A^\pi(s,a)$	the **advantage function**.

The first line says $V^\pi(s)$ is simply the average of $Q^\pi(s,a)$ over whichever action π happens to pick — which makes sense, because "follow π starting at s" literally means "let π choose an action, then keep going." The second line, the advantage, answers one very specific question: how much better (or worse) is it to take this particular action a, compared to just letting the policy do whatever it would normally do here? A positive advantage means a beats the policy's usual choice; a negative advantage means a is worse than the policy's usual choice.

2.2 A worked example (Slide 3)

A person can choose a_1 (practice drums), a_2 (watch TV), or a_3 (rest). The reward is 1 if they can play the instrument within a month, and 0 otherwise. Right now, the policy always chooses a_1: $\pi(a_1 \mid s) = 1$ in every state s.

Let's reason through what V, Q, and A mean here. Since the current policy *always* picks a_1, there is nothing else to average over, so $V^\pi(s)$ is just equal to $Q^\pi(s,a_1)$. That immediately tells us

$A^\pi(s, a_1) = Q^\pi(s, a_1) - V^\pi(s) = 0$. This is a useful general fact to remember: for any policy that always picks the same fixed action, the advantage of that very action is always exactly zero. For the other two actions, which the current policy never picks, $Q^\pi(s, a_2)$ and $Q^\pi(s, a_3)$ ask a "what if" question: what if I did this once, and then went right back to practicing drums afterward? Since skipping a day of practice probably doesn't help (and likely hurts) the chance of learning the instrument in a month, we'd expect $Q^\pi(s, a_2)$ and $Q^\pi(s, a_3)$ to be no better than $Q^\pi(s, a_1)$ — giving those actions a zero or negative advantage. The point of this example isn't to get one exact number; it's to practice thinking through what V, Q, and A actually mean for a real policy.

2.3 Issues with vanilla policy gradient, recapped (Slide 4)

This slide reminds us of Lecture 06's problems, using the same walking and jacket-folding examples. Reward-to-go can still push down the likelihood of a genuinely good early action (like a step forward) if the run ends badly overall, because reward-to-go is still just one noisy sample of what happened next, not a trustworthy measure of how good that action really was on average. And with sparse rewards, most sampled runs (like "folds only the sleeves" or "flattens the jacket but doesn't fold it") don't get used efficiently, because their reward-to-go values all look almost the same. The slide sums this up in its own words: *"Doesn't make efficient use of data! Can we learn what is good & bad?"* The rest of the lecture answers that question by learning a critic.

2.4 The conceptual pivot: reward-to-go and baseline are secretly Q and V (Slide 5)

$$\sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_t, a_t] = Q^\pi(s_t, a_t), \qquad b = \text{average reward} = \frac{1}{N} \sum_i Q^\pi(s_{i,t}, a_{i,t}) = V^\pi(s_t),$$

$$\nabla_\theta J(\theta) \approx \sum_i \sum_{t=1}^{T} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i)\, A^\pi(s_{i,t}, a_{i,t}).$$

The first equation says: if you could somehow compute the *true, exact* average of the reward-to-go instead of just one noisy sample of it, you would get precisely $Q^\pi(s_t, a_t)$. The second equation says: if you average those Q^π values (equivalently, average many reward-to-go samples) over many actions the policy takes, you get exactly $V^\pi(s_t)$.

This is arguably the single most important idea in the whole lecture, so it's worth spelling out slowly. Lecture 06's "reward-to-go" was really always just a rough, one-sample guess at $Q^\pi(s, a)$, and its "average-return baseline" was really always just a rough guess at $V^\pi(s)$. That means the whole "reward-to-go minus baseline" quantity from Lecture 06 was secretly, all along, a noisy way of estimating the advantage $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$. Lecture 07's big idea is: instead of guessing Q and V crudely from just one sampled run, why not actually *learn* them properly, using a trained function for each one? Then we plug the resulting, much less noisy advantage straight into the same policy gradient formula. This is the bridge from "policy gradient" to "actor-critic": the policy is the **actor**, and the learned V (or Q) is the **critic** grading it.

2.5 The general actor-critic loop (Slide 6)

This slide uses the same gradient formula as above, wrapped inside a repeating loop: start with some policy (random, from imitation learning, or from hand-made rules), and then keep repeating: collect data, update the critic, and update the actor using the critic's advantage estimates.

2.6 Estimating the advantage with one bootstrapped sample (Slide 7)

Starting from the exact relationship $Q^\pi(s_t, a_t) = \sum_{t'=t}^{T} \mathbb{E}_{\pi_\theta}[r(s_{t'}, a_{t'}) \mid s_t, a_t] = r(s_t, a_t) + \mathbb{E}_{s_{t+1}\sim p(\cdot|s_t,a_t)}[V^\pi(s_{t+1})]$, this slide replaces the inner average with just one sampled next state, giving a shortcut estimate:

$$Q^\pi(s_t, a_t) \approx r(s_t, a_t) + V^\pi(s_{t+1}), \qquad A^\pi(s_t, a_t) \approx r(s_t, a_t) + V^\pi(s_{t+1}) - V^\pi(s_t).$$

Symbol	What it means
$r(s_t, a_t)$	the actual reward you received on this step.
$V^\pi(s_{t+1})$	the critic's own guess at how good the state you actually landed in is.
$V^\pi(s_t)$	the critic's guess at how good the state you started this step in was.

In plain words: instead of waiting until the entire rest of the run is finished to find out how good an action really was (as reward-to-go does), we take a shortcut. We assume the critic's guess, $V^\pi(s_{t+1})$, already correctly sums up everything good that will happen from here on, and we just add the one real reward we actually saw this step. Then we subtract off what we expected the starting state to be worth. This turns a "wait for the whole run to finish" estimate into a "just look one step ahead" estimate, called **bootstrapping** — we lean on our own critic's guess instead of waiting for the real outcome to arrive. The slide mentions two ways to train $V^\pi(s_t)$: Gradient Monte Carlo (train it to match the real, fully-summed future reward) or Semi-Gradient TD (train it to match this same one-step shortcut target). One thing worth flagging: elsewhere in the course, this same quantity includes a discount factor γ (a number between 0 and 1 that makes future rewards count for slightly less than reward right now), written $\delta_t = r_t + \gamma\hat{V}(s_{t+1}) - \hat{V}(s_t)$ and called the **TD error**. This particular slide simply leaves γ out (which is the same as saying $\gamma = 1$). Either way, remember this: the one-step advantage *is* the TD error — there is nothing extra to compute beyond this one subtraction.

2.7 The actor-critic algorithm (Slide 8)

1. Run the current policy π_θ to collect a batch of data, $\{(s_{1,i}, a_{1,i}, \ldots, s_{T,i}, a_{T,i})\}$.
2. Train the critic, $\hat{V}_\phi^{\pi_\theta}$, so its guesses match the summed rewards seen in the data. (Here ϕ is just the critic's own set of dial-settings, kept separate from the policy's dials θ.)
3. Work out the advantage for each step: $\hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}) = r(s_{i,t}, a_{i,t}) + \hat{V}_\phi^{\pi_\theta}(s_{i,t+1}) - \hat{V}_\phi^{\pi_\theta}(s_{i,t})$.
4. Work out the gradient: $\nabla_\theta J(\theta) \approx \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i)\, \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t})$.
5. Update the policy's dials: $\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta)$, where α is again the learning rate — the small step size.

Aspects of "actor-critic."

- **What it is:** a policy gradient method where a separate, learned function (the critic) works out the advantage, instead of the rough Monte-Carlo reward-to-go-minus-baseline trick from Lecture 06.

- **Plain explanation:** think of it like a student (the actor) practicing a skill, while a coach (the critic) watches and gives feedback on whether each attempt was better or worse than expected — using the coach's own experience-based judgment, instead of waiting to see the final result of the whole competition.
- **Advantage:** much less wobbly than pure Monte-Carlo policy gradient, since the critic leans on its own prediction instead of one single noisy full-run sample. It also works even when rewards are sparse or delayed, since the critic can hand out useful partial credit along the way.
- **Disadvantage:** it introduces some bias, because the critic's guesses are themselves imperfect, especially early in training. It also means training and maintaining a second learned model (the critic), which adds complexity and its own possible mistakes.
- **Problem it solves:** the wobbly-gradient, poor-credit-assignment, and sparse-reward problems that hurt vanilla policy gradient in Lecture 06.
- **Problem it cannot solve on its own:** it still nominally needs fresh, on-policy data for the formula to be exactly correct — just like REINFORCE.
- **Problems it gives rise to:** trying to make it off-policy (to reuse old data) reopens the same importance-sampling and overfitting issues from Lecture 06, plus a brand-new question: which policy was the critic even trained for?
- **When to use it:** nearly always preferred over plain REINFORCE, especially for long tasks or ones with sparse rewards.
- **When not to use it:** in tasks so short and simple that the critic's extra bias and complexity aren't worth the trouble — plain Monte Carlo policy gradient may be simpler there.
- **What would change if done differently:** if the critic were trained using values from a stale or wrong policy, its advantage estimates (and therefore the actor's whole gradient) would be systematically wrong. This exact mistake is what breaks the naive replay-buffer version discussed a few slides ahead.

2.8 Off-policy actor-critic, version 1: multiple gradient steps (Slide 9)

1. Run the current policy π_θ to collect a batch of data.
2. Train the critic $\hat{V}_\phi^{\pi_\theta}$ to match the summed rewards.
3. Work out the advantage: $\hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}) = r(s_{i,t}, a_{i,t}) + \hat{V}_\phi^{\pi_\theta}(s_{i,t+1}) - \hat{V}_\phi^{\pi_\theta}(s_{i,t})$.
4. Work out the gradient: $\nabla_{\theta'} J(\theta') \approx \sum_i \sum_t \dfrac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_t^i \mid s_t^i)\, \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t})$, using the same importance ratio idea from Lecture 06.
5. Update the policy's dials: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

This is exactly Lecture 06's per-timestep importance-sampling fix, now applied on top of the learned advantage instead of the raw reward-to-go. The advantage values used here still describe the *old* policy that produced the data; multiplying by the importance ratio boosts the probability of actions that had a high advantage under that old policy. The slide asks a fair question: what could go wrong if we take too many gradient steps on this same one batch of data?

2.9 What breaks with too many gradient steps, and two fixes (Slide 10)

Here's the problem: the policy is pushed to become more and more different from the old policy that produced the advantage estimates, and it can easily start overfitting to just this one batch of data — like memorizing the answers to one specific practice test instead of actually learning the subject.

The first fix is to reuse the KL idea from Lecture 06: $\mathbb{E}_{s\sim\pi_\theta}[D_{KL}(\pi_{\theta'}(\cdot \mid s) \,\|\, \pi_\theta(\cdot \mid s))] \le \delta$ — keep a leash on how far the new policy is allowed to wander from the old one. (This exact same idea, applied to language models, is the basis for some LLM preference-tuning methods.)

The second fix, instead of limiting the KL divergence directly (which is expensive and fiddly to enforce exactly), is simpler: just put a cap on how large the importance ratio itself is allowed to get. This doesn't directly force the two policies to be close, but it removes the temptation to push the ratio far outside a safe range. This is the key idea behind Proximal Policy Optimization, or **PPO**.

2.10 The surrogate objective (Slide 11)

$$\nabla_{\theta'} J(\theta') \approx \sum_i \sum_t \frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})} \nabla_{\theta'} \log \pi_{\theta'}(a_t^i \mid s_t^i)\, \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}).$$

Using the same rearranging trick from Lecture 06 (that $p_\theta(\tau)\nabla_\theta \log p_\theta(\tau) = \nabla_\theta p_\theta(\tau)$, run in reverse), this gradient turns out to be exactly the gradient of a simpler-looking single number, called the **surrogate objective**:

$$\tilde{J}(\theta') \approx \sum_{t,i} \frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})} \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}).$$

Symbol	What it means
$\tilde{J}(\theta')$	just one number we can directly try to make as large as possible, using any standard optimizer, without ever manually writing out the "log-probability times advantage" gradient formula ourselves — its gradient, computed at $\theta' = \theta$, automatically gives back the exact actor-critic gradient shown above.

In plain words: instead of hand-coding the "make good actions more likely" gradient formula, we can just hand this simple ratio-times-advantage number to an automatic optimizer, and pushing it up will naturally increase the probability of high-advantage actions. But the catch, again: this shortcut number is only a trustworthy stand-in for the real objective J when θ' stays close to θ. If θ' is allowed to wander too far while chasing a bigger $\tilde{J}(\theta')$, the policy is again tempted to overfit to this one batch, exactly like before.

2.11 PPO Trick #1 and #2: clip and take the minimum (Slide 12)

The first trick is to cap, or "clip," the ratio:

$$\tilde{J}(\theta') \approx \sum_{t,i} \text{clip}\left(\frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})},\, 1-\epsilon,\, 1+\epsilon\right) \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}).$$

Symbol	What it means
$\text{clip}(x, 1-\epsilon, 1+\epsilon)$	if x already sits between $1-\epsilon$ and $1+\epsilon$, leave it alone; if x is bigger than $1+\epsilon$, replace it with $1+\epsilon$; if x is smaller than $1-\epsilon$, replace it with $1-\epsilon$.
ϵ	just a small number (a common choice is 0.2) that controls how far the ratio is allowed to move before it gets clamped down.

In plain words: this directly caps how much extra credit the objective can hand out for an action whose new-policy probability has drifted very far from its old-policy probability. Once the ratio is outside the allowed band, pushing it further doesn't increase the objective anymore, which removes the temptation to keep pushing it that way.

The second trick is to take the smaller (the minimum) of the clipped and unclipped versions:

$$\tilde{J}(\theta') \approx \sum_{t,i} \min\left(\frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})} \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}),\ \ \text{clip}\left(\frac{\pi_{\theta'}(a_{i,t} \mid s_{i,t})}{\pi_\theta(a_{i,t} \mid s_{i,t})},\, 1-\epsilon,\, 1+\epsilon \right) \hat{A}^{\pi_\theta}(s_{i,t}, a_{i,t}) \right).$$

Part of the formula	What it means
first term inside $\min(\cdot,\cdot)$	the plain, unclipped version from before.
second term inside $\min(\cdot,\cdot)$	the clipped version we just built.

This is worth understanding, not just memorizing. Clipping by itself only works in one direction. Imagine a bad action, where the advantage $\hat{A}$ is negative, and its ratio has already dropped *below* $1-\epsilon$ (meaning the new policy has already strongly moved away from this bad action — which is a good thing!). Without the minimum, clipping that ratio back up to $1-\epsilon$ would actually make the objective's value for that term *less negative* than the true, unclipped number — which secretly hides just how far the policy has already correctly moved away from the bad action, and could even create a temptation to drift back toward it. Taking the minimum of the clipped and unclipped terms fixes this: it makes the objective always a "worst-case, no cheating" estimate, so the policy is never rewarded for having drifted further in a harmful direction — only ever held back or left alone. As a quick worked check: with ratio $r = 1.5$, advantage $\hat{A} = +8$, and $\epsilon = 0.2$, the unclipped term is 12.0, and the clipped term is $\text{clip}(1.5, 0.8, 1.2) \times 8 = 9.6$. The minimum of the two is 9.6, so the clipped term is the one that's actually used — telling us the update is being held back because the ratio moved outside the trusted zone.

2.12 PPO Trick #3: Generalized Advantage Estimation, GAE (Slide 13)

$$\hat{A}^\pi_n(s_t, a_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}, a_{t'}) - \hat{V}^\pi_\phi(s_t) + \gamma^n \hat{V}^\pi_\phi(s_{t+n}), \qquad \hat{A}^\pi_{GAE}(s_t, a_t) = \sum_{n=1}^{\infty} w_n \hat{A}^\pi_n(s_t, a_t),\ w_n \propto \lambda^{n-1}.$$

Symbol	What it means
$\hat{A}_n^\pi$	called an n-step advantage: it uses n real, actually-observed rewards, each shrunk slightly by the discount factor $\gamma^{t'-t}$ depending on how far in the future it happened, and then it bootstraps (leans on the critic's prediction) for whatever state is reached n steps later.
γ	a number between 0 and 1 that says how much future rewards are worth compared to reward right now.
$\hat{A}_{GAE}^\pi$	blends together the n-step advantages for every possible value of n at once, each one multiplied by a weight w_n that shrinks smaller and smaller as n grows, controlled by a dial λ between 0 and 1.

Setting $n = 1$ inside $\hat{A}_n^\pi$ gives back exactly the one-step TD error δ_t from a few slides ago — so GAE is really just a bigger, more flexible version of what we already had. Using small n (or, equivalently, a small λ, since that makes the weights shrink fast so only the small-n terms matter much) means we trust the critic's own guess more, and rely less on real, noisy sampled rewards: this gives lower wobble but more built-in bias, since we're leaning heavily on a critic that might not be perfect. Using large n (or a large λ) means we wait for more real rewards before leaning on the critic, getting closer to a plain full-run estimate: this gives lower bias but more wobble, since we're now trusting more noisy real samples and less of the (possibly flawed) critic. The slide's own phrase for this is: *"cut earlier for less variance."* In short, GAE gives us one single dial, λ, that smoothly trades off "trust the critic" against "trust the real rewards," instead of forcing us to pick one extreme or the other.

Aspects of "PPO and GAE."

- **What it is:** an off-policy actor-critic method that (1) clips the importance ratio, (2) takes the minimum of the clipped and unclipped terms, and (3) uses GAE to estimate the advantage.
- **Advantage:** it allows several gradient steps per batch of data (better use of data than a plain, on-policy actor-critic), while being much simpler to build than an exact KL-limited method, since clipping needs no extra fancy math tools.
- **Disadvantage:** it only removes the *temptation* to move far, it does not *guarantee* a hard limit the way an exact KL rule would. Clipping is a rough trick, so its behaviour right at the edge of the clip can feel a bit arbitrary. It is still fundamentally a "stay close" method, not a truly off-policy one.
- **Problem it solves:** the "too many gradient steps causes overfitting" problem from a couple of slides earlier, without needing to compute or enforce a KL limit exactly.
- **Problem it cannot solve:** data that comes from a policy very far from the current one. **PPO only works well when the two policies are close together (it's a "proximal" method, not a fully off-policy one).** Its surrogate objective is only a good stand-in for the real objective when $\pi_{\theta'}$ is near π_θ. With very old replay-buffer data, the ratio would sit far outside $[1-\epsilon, 1+\epsilon]$, so clipping just zeroes out the gradient for those samples entirely — no learning signal at all, rather than any useful correction.

- **Problem it gives rise to:** since clipping only holds back the gradient rather than truly making old data usable, learning from a large, old pool of past experience needs an entirely different tool — learned Q-functions with freshly resampled actions, covered a few slides ahead — instead of stronger importance-sampling correction.
- **When to use it:** as the standard, practical choice for most deep RL settings with moderate reuse of each batch.
- **When not to use it:** with a big replay buffer full of data from very old policies.
- **What would change if done differently:** clipping without also taking the minimum would let the objective be tricked for bad actions whose ratio already collapsed low, as explained above; using $n = \infty$ (equivalent to $\lambda = 1$, pure full-run estimation) in GAE would give the high-wobble, low-bias extreme; using $n = 1$ (equivalent to $\lambda \to 0$) gives back the low-wobble, high-bias one-step TD extreme.

2.13 The full PPO algorithm (Slide 14)

1. Run the current policy π_θ to collect a batch of data.
2. Train the critic $\hat{V}_\phi^{\pi_\theta}$ to match the summed rewards in the data.
3. Work out $\hat{A}^\pi_{GAE}(s_t, a_t)$, as described above.
4. Update the policy with M gradient steps on the clipped-and-minimum surrogate objective from the previous slide.

This simply puts all three tricks — clip, take the minimum, and GAE — together into one practical loop, and it explicitly allows $M > 1$ gradient steps per collected batch, which is the entire point of building the clip-and-minimum trick in the first place.

2.14 KL constraint's $\delta \to 0$ and $\delta \to \infty$ behaviour

This isn't its own separate slide, but it's a natural and useful extension of the earlier KL-limit idea to keep in mind alongside PPO. As $\delta \to 0$, the limit forces the new policy $\pi_{\theta'}$ to stay basically identical to the old policy π_θ — learning is effectively frozen: extremely safe, but zero progress. As $\delta \to \infty$, the limit stops mattering at all, and the method just becomes the unconstrained off-policy update from a few slides earlier, where the policy is free to overfit to the batch and the importance ratios become unreliable. PPO's clipping is the "soft," easy-to-build version of this exact same idea — worth naming directly if a question asks for the method that caps importance ratios instead of limiting the KL divergence.

2.15 Motivation for full off-policy learning: replay buffers (Slide 15)

So far, being "off-policy" has only meant "squeeze a few extra gradient steps out of the current batch." Here's the next natural question: could we reuse *all* of the experience we've ever collected, no matter how old it is? Two ingredients make this possible: first, keep a big storage box (called a **replay buffer**) holding every past (s, a, r, s') example we've ever seen; second, rewrite the equations so they no longer secretly assume the data is fresh and on-policy.

2.16 Off-policy actor-critic v2, naive attempt with V (Slide 16)

1. Run the current policy π_θ to collect experience $\{s_i, a_i\}$; add it into the replay buffer.
2. Train the critic $\hat{V}_\phi^\pi$ using the summed rewards seen in the sampled data.
3. Work out the advantage: $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \hat{V}_\phi^\pi(s_i') - \hat{V}_\phi^\pi(s_i)$.
4. Work out the gradient: $\nabla_\theta J(\theta) \approx \sum_i \nabla_\theta \log \pi_\theta(a_i \mid s_i)\, \hat{A}^\pi(s_i, a_i)$ — but now computed using a random mini-batch pulled from *all* of the past data in the buffer, not just the newest batch.
5. Update the dials: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

On the surface, this looks like a harmless upgrade to the actor-critic algorithm — just sample from a bigger pool of data (the whole buffer) instead of only the newest batch. But it quietly carries over the same on-policy assumptions from before, and those assumptions turn out to be broken here, as the next slide explains.

2.17 Diagnosing what breaks with a naive replay buffer (Slide 17)

1. Run the current policy π_θ to collect experience $\{s_i, a_i\}$; add it to buffer R.
2. Sample a mini-batch $\{s_i, a_i, r_i, s_i'\}$ from R.
3. Update the critic $\hat{V}_\phi^\pi$ using the target $y_i = r_i + \gamma \hat{V}_\phi^\pi(s_i')$ for each s_i (a target is just the number we train the critic to try to match).
4. Work out the advantage: $\hat{A}^\pi(s_i, a_i) = r(s_i, a_i) + \hat{V}_\phi^\pi(s_i') - \hat{V}_\phi^\pi(s_i)$.
5. Work out the gradient: $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(a_i \mid s_i)\, \hat{A}^\pi(s_i, a_i)$.
6. Update the dials: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

This slide asks the key diagnostic question: when we train V^π using data pulled from the whole replay buffer — data that was actually collected by many different, older versions of the policy — which policy π is this critic actually learning a value function for? The answer given right on the slide: *"some kind of average"* — a fuzzy mash-up of whatever old policies happened to generate the buffer's data, not the current policy π_θ we actually care about.

Two specific things are broken here, and both are worth remembering exactly:

1. The training target $y_i = r_i + \gamma \hat{V}_\phi^\pi(s_i')$ is not correctly estimating V^{π_θ} at all — it's contaminated by a blend of whatever old policies filled up the buffer.
2. The current policy π_θ probably would not have chosen action a_i in state s_i in the first place, since that action actually came from some old, different policy. So $\nabla_\theta \log \pi_\theta(a_i \mid s_i)$ ends up being computed using the "wrong" likelihood — an action the current policy considers unlikely, or would never pick — which throws off the direction of the whole gradient.

2.18 The fix, part 1: fit Q instead of V (Slide 18)

The question this slide asks: how do we train a value function for the *current* policy π_θ, using a replay buffer full of data from *past*, different policies? The answer: train $Q^{\pi_\theta}(s,a)$ instead of $V^{\pi_\theta}(s)$, using this relationship:

$$Q^{\pi_\theta}(s,a) = r(s,a) + \gamma\, \mathbb{E}_{s'\sim p(\cdot|s,a),\, \bar{a}'\sim\pi_\theta(\cdot|s')}\big[Q^{\pi_\theta}(s',\bar{a}')\big],$$

approximated in practice by: first, sample (s_i, a_i, s_i') from the buffer; second, sample a fresh action $\bar{a}_i' \sim \pi_\theta(\cdot \mid s_i')$ *from the current policy itself* (not read out of the buffer), giving the training target

$$Q^{\pi_\theta}(s_i,a_i) \approx r(s_i,a_i) + \gamma\, Q^{\pi_\theta}(s_i',\bar{a}_i') =: y_i.$$

Symbol	What it means
(s_i, a_i, s_i')	come straight out of the buffer — they're just facts about what actually happened in the world, and they stay true no matter which policy caused them.
$\bar{a}_i'$	a brand-new, made-up action, generated right now by asking the *current* policy π_θ what it would do in state s_i' — it is not the action that was actually taken next in the stored data.

In plain words: the pair (s_i, a_i) is a historical fact — it doesn't matter which policy caused it, the reward $r(s_i, a_i)$ and the resulting next state s_i' are still true, useful facts about the world. What we must *not* reuse from the buffer is what happened right after s_i', because that choice was made by an old policy we no longer care about. Instead, we ask the current policy what it would do at s_i' and use its answer to build the training target. That way, $Q^{\pi_\theta}(s_i, a_i)$ ends up being trained to reflect the value of taking action a_i and then following the *current* policy afterward — which is exactly the number we want.

2.19 Assembling the fixed algorithm (Slides 19–20)

1. Run the current policy π_θ to collect experience $\{s_i, a_i\}$; add it to buffer R.
2. Sample a mini-batch $\{s_i, a_i, r_i, s_i'\}$ from R.
3. Train $\hat{Q}^\pi_\phi$ using the target $y_i = r(s_i,a_i) + \gamma\, \hat{Q}^\pi_\phi(s_i', a_i')$, where $a_i' \sim \pi_\theta(\cdot \mid s_i')$ is a fresh action sampled from the current policy, not read from the buffer.
4. (Two more small design changes, explained just below.)
5. Update the dials: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

These slides make two further changes explicit:

The first change is to drop the baseline and use $\hat{Q}$ directly, instead of building a separate advantage using an average-reward baseline. This makes the gradient wobblier (since there's no baseline being subtracted anymore), but it's an acceptable trade-off, since we're now using far more data — the entire replay buffer — which helps steady things out in a different way.

The second change is to also resample the action used inside the actor's own gradient formula. The current policy's actions are probably better than whatever some old, past policy happened to do, so instead of reusing the buffer's stored action a_i, we sample a fresh action from the current policy, $\bar{a}_i \sim \pi_\theta(\cdot \mid s_i)$, and use it here:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\bar{a}_i \mid s_i)\, \hat{Q}^\pi(s_i, \bar{a}_i).$$

2.20 The final off-policy actor-critic algorithm (Slide 21)

1. Run the current policy π_θ to collect experience $\{s_i, a_i\}$; add it to buffer R.
2. Sample a mini-batch $\{s_i, a_i, r_i, s'_i\}$ from R.
3. Train $\hat{Q}^\pi_\phi$ using the target $y_i = r(s_i, a_i) + \gamma\, \hat{Q}^\pi_\phi(s'_i, a'_i)$, where $a'_i \sim \pi_\theta(\cdot \mid s'_i)$.
4. Work out the gradient: $\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_i \nabla_\theta \log \pi_\theta(\bar{a}_i \mid s_i)\, \hat{Q}^\pi(s_i, \bar{a}_i)$, where $\bar{a}_i \sim \pi_\theta(\cdot \mid s_i)$.
5. Update the dials: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.

Why resample $\bar{a}$ from π_θ instead of using the buffer's action? This is a design question worth being able to answer, not just a step to memorize. The buffer's stored action a_i came from some *old*, past behaviour policy. If we used a_i directly inside the actor's gradient, we would end up training toward the value of that *old* policy, not the current one. By resampling $\bar{a}_i \sim \pi_\theta(\cdot \mid s_i)$ instead, $\hat{Q}^{\pi_\theta}(s_i, \bar{a}_i)$ becomes a genuinely accurate estimate of what the *current* policy would actually get. This makes the gradient $\nabla_\theta \log \pi_\theta(\bar{a}_i \mid s_i)\, \hat{Q}^{\pi_\theta}(s_i, \bar{a}_i)$ correctly point toward improving π_θ, and it removes any need for importance ratios entirely — notice there is no ratio of probabilities anywhere in this final formula. There is a trade-off worth remembering for later: this exact trick — asking the current policy for a made-up action, and then trusting $\hat{Q}$'s guess about it, even though that action never actually happened in the real data — is precisely what becomes risky once there's no way to go out and collect more real data to double-check the answer (this is called the "out-of-distribution action" problem, and it's central to *offline* RL in Lecture 08). In an online setting, like here, it's safe, because the agent can always go try more things in the real world to correct any mistake $\hat{Q}$ makes about that made-up action.

Aspects of "off-policy actor-critic with a replay buffer and Q-function."

- **What it is:** an actor-critic method that stores all past experience in a replay buffer, trains Q^{π_θ} (instead of V^{π_θ}) using targets built from freshly resampled current-policy actions, and updates the actor using those same resampled actions — with no importance ratios anywhere.
- **Advantage:** it lets us truly reuse an unlimited amount of old data, not just "a few extra gradient steps" like PPO. It also completely avoids the exploding or vanishing importance-ratio problem, since no ratio is needed at all.
- **Disadvantage:** it's wobblier than the baseline-subtracted version, since no $V(s)$ baseline is subtracted anymore. It also depends on $\hat{Q}$ being accurate even for freshly made-up actions that the buffer never actually contains — and if $\hat{Q}$ is wrong there, the agent has no easy way to notice, other than actually trying it.
- **Problem it solves:** both problems diagnosed a few slides earlier (a training target contaminated by a mix of old policies, and a gradient computed at the wrong, stale action). It also lets us use arbitrarily old, off-policy data, which PPO's clipping trick simply cannot do.
- **Problem it cannot solve:** it does not work at all if the agent has no way to go out and collect new real experience to catch and correct $\hat{Q}$'s mistakes on made-up, resampled actions — that exact failure is what makes this same-looking trick dangerous in pure offline RL.

- **When to use it:** in online settings with an expensive-to-collect but reusable stream of experience — robots, simulators, and so on — where every past interaction should count for as much as possible.
- **When not to use it:** in a purely offline setting, with a fixed dataset and no way at all to gather new corrective experience — there, guessing Q at unseen, made-up actions is unsafe, which is exactly why Lecture 08's offline RL methods use very different machinery.
- **What would change if done differently:** if the action a'_i used for the Q-target, or the action $\bar{a}_i$ used in the actor's gradient, were instead pulled straight from the replay buffer rather than resampled from π_θ, the algorithm would quietly fall right back into the two broken behaviours diagnosed earlier — learning values and gradients for the wrong, old, blended policy instead of the current one.

3 The single thread connecting Lectures 06 and 07

If you remember only one sentence from these two lectures, make it this one: **every algorithm here fixes one specific problem in the algorithm before it, and every fix creates a brand-new problem of its own.** Here is that chain, step by step:

- Vanilla policy gradient: simple, but very wobbly (high variance).
- $\rightarrow$ reward-to-go: fixes bad credit-assignment, but is still noisy.
- $\rightarrow$ baseline: cuts down the wobble, but still needs completely fresh, on-policy data.
- $\rightarrow$ importance sampling: lets us reuse a bit of old data, but the correction explodes or vanishes for long runs.
- $\rightarrow$ KL constraint: keeps the new policy close to the old one, but is hard to enforce exactly.
- $\rightarrow$ PPO clipping: easy to build and use, but only trustworthy when the two policies are already close together.
- $\rightarrow$ replay buffer + learned Q-function: fully reuses old data, but has to trust guesses about made-up actions — which is only safe because, in this online setting, we can always go collect more real data to check those guesses.

Almost every "explain" or "diagnose" style question about these two lectures comes down to naming which link in this chain is being tested, and then stating both halves of the story: the problem being fixed, and the brand-new limitation that the fix itself introduces.